

系统底层架构深度拆解

DeepSeek Harness

基于 Cordis 微内核的工业级开源智能体运行时

Cyber·X·Lab 硬科技研发与系统工程解析 | 开源布道讲义

传统 Agent 框架的生产级瓶颈

✖ 01

上下文隐式污染

扁平全局字典无边界传递。执行超过 20 步后，前期分支脏变量滞留，持续毒化模型后续推理。

✖ 02

进程树泄漏与孤儿进程

终端命令被视作普通 RPC。智能体崩溃时，后台编译器/开发服务器失控，占用端口与内存。

✖ 03

静态 DAG 拓扑死锁

机制与策略强耦合。工具调度与固定有向图绑定，一旦需要现场动态挂载插件，必须重构整张图。

系统核心目标与显式非目标

核心架构目标 (Goals)

- **基于 Cordis 微内核实现“万物皆插件”**：解耦插槽化依赖注入
- **统一多端宿主形态**：一套内核无缝驱动 Web UI (3080)、Desktop、CLI 与 SDK
- **确定性副作用治理**：引入代数效果流与对偶补偿回滚机制
- **操作系统级进程硬隔离**：POSIX 独立进程组彻底消除孤儿进程

显式非目标 (Non-Goals)

- **不绑定专有闭源模型**：模型路由层通用适配 DeepSeek 官方及任何兼容端点
- **不强行内置重型容器**：内核不捆绑 Docker 守护进程，交由宿主轻量容器契约
- **不做黑盒单体 RAG**：解耦检索为纯插件能力，拒绝框架死板捆绑

物理与逻辑分层全景

宿主交互层

统一多端客户端

- **Web UI**: 默认端口 3080 看板
- **Desktop**: Electron 桌面工作区
- **CLI & SDK**: 命令行与 Python 绑定

DSH CORE 内核

Cordis 微内核

- 事件调度总线
- **Context** 依赖注入容器
- 热重载插件服务网格

环境执行层

物理边界隔离

- **POSIX** 独立进程组
- 虚拟文件系统沙盒
- 状态变更追溯与回滚账本

客户端通过标准 JSON-RPC / IPC 与微内核通信，内核协调各插件完成受控环境调度。

Cordis 微内核：控制反转与插件契约

插件生命周期契约

所有业务能力（模型路由、工具沙盒、会话管理）均以独立生命周期挂载：

- `apply(ctx)`：声明服务契约与依赖注入
- 严格通过 Context 请求与提供能力
- 杜绝全局可变状态与模块越权

Creator 模式：现场动态扩容

遇到未知工具需求时，智能体自主编写并挂载插件：

- 智能体即时编写插件 TypeScript 源码
- 内存中即时编译、类型校验与沙盒验证
- 零停机动态热加载进微内核生效

可逆代数效果流与对偶补偿机制

确定性环境状态治理

禁止将对外部环境的修改视作无序副作用。每一个操作均携带严格的代数对偶补偿契约：

1. DISPATCH

正向操作执行

执行文件写入、终端指令或网络交互前，执行物理快照。

2. AUDIT

效果账本提交

将状态增量写入仅追加（Append-Only）审计日志。

3. ROLLBACK

对偶对冲回滚

出现异常或用户撤销时，自动逆向对偶执行，无脏状态残留。

POSIX 进程树隔离与孤儿进程彻底消除

传统框架的僵尸子进程痛点

当智能体执行 `npm run dev` 或重编译时，普通的 `child.kill()` 仅杀死了最外层的 Shell 包装：

✗ 深层后台服务器持续在操作系统后台泄漏，占用系统端口与内存。

DSH 独立进程组契约

- 启动进程时调用 `setsid()` 创建独立会话
- 所有衍生子进程严格继承负 PGID
- 清理时全量广播信号： `kill(-pgid, SIGTERM)`
- 彻底消除孤儿进程，重启后环境零泄漏

一套内核驱动多端宿主形态

Web UI 交互看板

内置 Vite 仪表盘，默认监听

`127.0.0.1:3080`，提供可视化单步调试、工具调用审计与会话时光机回溯。

Desktop 桌面客户端

基于 Electron 打造，深度集成操作系统通知中心、本地工作区拖拽挂载与本地加密钥匙串管理。

Headless CLI & Python SDK

轻量单一二进制与 Python 绑定，专为 CI/CD 自动化流水线、SWE-bench 基准评测及服务器调度设计。

同一套底层微内核与协议：杜绝多端状态分裂与接口不一致。

架构代价与已知妥协 (Architectural Taxes)

~3-5%

调度延迟代价

通过 Cordis 事件钩子进行动态上下文注入，相比单体硬编码调用有微秒级的微小开销。

I/O

快照账本存储代价

维护完整可逆代数效果流需要持久化本地磁盘日志，在高频大步骤下产生本地 I/O 写入。

POSIX

跨平台模拟代价

负 PGID 进程组清理逻辑在 Windows 原生内核上需特殊封装 Job Object 进行兼容对齐。

选型决策树：何时采用 vs 何时切勿使用

✅ 强烈推荐使用场景

- 长时多步骤软件研发与自治编码 (SWE-bench)
- 现场动态能力扩容需求 (Creator 模式)
- 跨 Web、桌面与 CI 统一驱动架构
- 对外部环境写操作有极高审计与回滚要求的系统

❌ 切勿盲目使用反模式

- 单次问答提示词聊天机器人
- 只读型纯文本检索 RAG 流程
- 超低延迟实时语音转录中继管道
- 极端短命瞬态无状态 Serverless Lambda (<500ms)

3 步生产环境快速启动

第 1 步

全局安装

通过 npm 安装 CLI:

```
npm i -g @dsh/cli
```

第 2 步

初始化工作区

初始化微内核配置:

```
dsh init --runtime cordis
```

第 3 步

启动服务

启动统一智能体宿主:

```
dsh serve --port 3080
```

验收验证: 浏览器打开 `http://localhost:3080` , 即可实时监控插件网格拓扑与智能体推理推演流。

开源布道与硬科技研究

Cyber·X·Lab

探索 AI 系统架构与主权工程的下一代范式

 官方门户
cyberxlab.xyz

 深度视频
YouTube @CyberXLab

 实时动态
X / Twitter @CyberXLab